

## Understanding the Linux File System

**Table 4-1 (continued)**

| <b>Directory</b> | <b>Contains</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>/media</b>    | The <b>/media/cdrom</b> or <b>/media/cdrom0</b> directory is for mounting the CD/DVD-ROM drive. If you have a CD/DVD recorder, you find a <b>/media/cdrecorder</b> directory instead of <b>/media/cdrom</b> and may also find <b>/media/ DVD</b> . Used for mounting file systems on removable media, such as CD/DVD-ROM drives, flash drives, external drives, floppy disks, and Zip drives. If you have a very old machine that still has a floppy drive on it, then the <b>/media/floppy</b> directory will also exist for mounting floppy disks. |
| <b>/mnt</b>      | Temporarily mounted file systems.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>/opt</b>      | Storage for large application software packages. For example, some distributions install the LibreOffice.org Office suite in the <b>/opt directory</b> .                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>/proc</b>     | Various information about the processes running in the Linux system.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>/root</b>     | Home directory for the <b>root</b> user.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>/sbin</b>     | Executable files representing commands typically used for system administration tasks and used by the <b>root</b> user. Commands such as <b>halt</b> and <b>shutdown</b> reside in the <b>/sbin</b> directory.                                                                                                                                                                                                                                                                                                                                       |
| <b>/srv</b>      | Data for services (such as web and FTP) offered by this system.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>/sys</b>      | Information about the devices, as seen by the Linux kernel.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>/tmp</b>      | Temporary directory that any user can use as a <i>scratch</i> directory, meaning that the contents of this directory are considered unimportant and usually are deleted every time the system boots.                                                                                                                                                                                                                                                                                                                                                 |
| <b>/usr</b>      | Subdirectories for many important programs, such as the X Window System (in the <b>/usr/X11R6</b> directory) and the online manual. (Table 4-2 shows some of the standard subdirectories in <b>/usr</b> .)                                                                                                                                                                                                                                                                                                                                           |
| <b>/var</b>      | Various system files (such as logs), as well as directories for holding other information, such as files for the web server and anonymous FTP server.                                                                                                                                                                                                                                                                                                                                                                                                |

The **/usr** and **/var** directories also contain a number of standard subdirectories. Table 4-2 lists the important subdirectories in **/usr**. Table 4-3 shows a similar breakdown for the **/var** directory.

### *Understanding the Linux File System*

**Table 4-2** Important **/usr** Subdirectories

| <i>Subdirectory</i>   | <i>Description</i>                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>/usr/bin</b>       | Executable files for many more Linux commands, including utility programs that are commonly available in Linux but aren't part of the core Linux operating system.                                                                                                                                                                                                 |
| <b>/usr/games</b>     | Some old Linux games.                                                                                                                                                                                                                                                                                                                                              |
| <b>/usr/include</b>   | Header files (filenames ending in <b>.h</b> ) for the C and C++ programming languages and the X11 header files in the <b>/usr/include/X11</b> directory and the Linux kernel header files in the <b>/usr/include/linux</b> directory.                                                                                                                              |
| <b>/usr/lib</b>       | Libraries for C and C++ programming languages and many other libraries, such as database libraries and graphical toolkit libraries.                                                                                                                                                                                                                                |
| <b>/usr/local</b>     | Local files. The <b>/usr/local/bin</b> directory, for example, is supposed to be the location for any executable program developed on your system.                                                                                                                                                                                                                 |
| <b>/usr/sbin</b>      | Many administrative commands, such as commands for e-mail and networking.                                                                                                                                                                                                                                                                                          |
| <b>/usr/share</b>     | Shared data, such as default configuration files and images for many applications. For example, <b>/usr/share/gnome</b> contains various shared files for the GNOME desktop, and <b>/usr/share/doc</b> has the documentation files for many Linux applications (such as the <b>bash</b> shell, the Sawfish window manager, and The GIMP image processing program). |
| <b>/usr/share/man</b> | Online manual (which you can read by using the <b>man</b> command).                                                                                                                                                                                                                                                                                                |
| <b>/usr/src</b>       | Source code for the Linux kernel (the core operating system).                                                                                                                                                                                                                                                                                                      |

**Table 4-3** Important **/var** Subdirectories

---

| <i><b>Subdirectory</b></i> | <i><b>Contains</b></i>                                                  |
|----------------------------|-------------------------------------------------------------------------|
| <b>/var/cache</b>          | Storage area for cached data for applications.                          |
| <b>/var/lib</b>            | Information relating to the current state of applications.              |
| <b>/var/lock</b>           | Locked files to ensure that a resource is used by one application only. |

*(continued)*

Table 4-3 (continued)

| Subdirectory            | Contains                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/var/log</code>   | Log files organized into subdirectories. The <b>syslogd</b> server stores its log files in <code>/var/log</code> , with the exact content of the files depending on the <b>syslogd</b> configuration file <code>/etc/syslog.conf</code> . For example, <code>/var/log/messages</code> is the main system log file; <code>/var/log/secure</code> contains log messages from secure services (such as <b>sshd</b> and <b>xinetd</b> ); and <code>/var/log/maillog</code> contains the log of mail messages. |
| <code>/var/mail</code>  | User mailbox files.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>/var/opt</code>   | Variable data for packages stored in <code>/opt</code> directory.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>/var/run</code>   | Data describing the system since the time it was booted.                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>/var/spool</code> | Data that's waiting for some kind of processing.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>/var/tmp</code>   | Temporary files preserved between system reboots.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>/var/yp</code>    | Network Information Service (NIS) database files.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

## Navigating the File System with Linux Commands

Although GUI file managers such as Nautilus (in GNOME) or Konqueror (in KDE) are easy to use, you can use them only if you have a working GUI desktop. Sometimes you may not have a graphical environment to run a graphical file manager. For example, you may be logged in through a text terminal, or X may not be working on your system. In those situations, you have to rely on Linux commands to work with files and directories. You can always use Linux commands, even in the graphical environment — all you have to do is open a terminal window and type the Linux commands.

In this section, I briefly describe some Linux commands for moving around the Linux file system.

### Commands for directory navigation

In Linux, when you log in as **root**, your home directory is `/root`. For other users, the home directory is usually in the `/home` directory. My home directory (when I log in as **edulaney**) is `/home/edulaney`. This information is stored in the `/etc/passwd` file. By default, only you have permission to save files in your home directory, and only you can create subdirectories in your home directory to further organize your files.

Linux supports the concept of a *current* directory, which is the directory on which all file and directory commands operate. After you log in, for example, your current directory is the home directory. To see the current directory, type the **pwd** command.

To change the current directory, use the **cd** command. To change the current directory to `/usr/lib`, type the following:

```
cd /usr/lib
```

Then, to change the directory to the **cups** subdirectory in `/usr/lib`, type this command:

```
cd cups
```

Now, if you use the **pwd** command, that command shows `/usr/lib/cups` as the current directory.

These two examples show that you can refer to a directory's name in two ways:

- ♦ **Absolute pathname:** An example is `/usr/lib`, which is an exact directory in the directory tree. Think of the absolute pathname as the complete mailing address for a package that the postal service will deliver to your next-door neighbor.

- ♦ **Relative directory name:** An example is **cups**, which represents the **cups** subdirectory of the current directory, whatever that may be. Think of the relative directory name as giving the postal carrier directions from your house to the one next door so the carrier can deliver the package.

If I type **cd cups** in **/usr/lib**, the current directory changes to **/usr/lib/cups**. However, if I type the same command in **/home/edulaney**, the shell tries to change the current directory to **/home/edulaney/cups**.



Use the **cd** command without any arguments to change the current directory back to your home directory. No matter where you are, typing **cd** at the shell prompt brings you back home!



The tilde character (**~**) refers to your home directory. Thus you can change the current directory to your home directory by using the command **cd ~**. You can refer to another user's home directory by appending that user's name to the tilde. Thus **cd ~superman** changes the current directory to the home directory of **superman**.



Wait, there's more. A single dot (**.**) and two dots (**..**) — often cleverly referred to as *dot-dot* — also have special meanings. A single dot (**.**) indicates the current directory, whereas two dots (**..**) indicate the parent directory. For example, if the current directory is **/usr/share**, you go one level up to **/usr** by typing the following: **cd ..**

## Commands for directory listings and permissions

You can get a directory listing by using the **ls** command. By default, the **ls** command — without options — displays the contents of the current directory in a compact, multicolumn format. For example, type the next two commands to see the contents of the **/etc/X11** directory:

```
cd /etc/X11 ls
```

The output looks like this. (On the console, you see some items in different colors.)

```
X Xsession.d cursors rgb.txt xkb
XF86Config-4 Xsession.options default-display-manager rstart xserver
Xresources Xwrapper.config fonts sysconfig xsm
Xsession app-defaults gdm xinit
```

From this listing (without the colors), you can't tell whether an entry is a file or a directory. To tell the directories and files apart, use the **-F** option with **ls**, like this:

```
ls -F
```

This time the output gives you some more clues about the file types:

```
X@ Xsession.d/ cursors/ rgb.txt xkb/
XF86Config-4 Xsession.options default-display-manager rstart/ xserver/
Xresources/ Xwrapper.config fonts/ sysconfig/ xsm/
Xsession* app-defaults/ gdm@ xinit/
```

The output from **ls -F** shows the directory names with a slash (/) appended to them. Plain filenames appear as is. The at sign (@) appended to a filename (for example, notice the file named **X**) indicates that this file is a link to another file. (In other words, this filename simply refers to another file; it's a shortcut.) An asterisk (\*) is appended to executable files. (**Xsession**, for example, is an executable file.) The shell can run any executable file.

You can see even more detailed information about the files and directories with the **-l** option:

```
ls -l
```

For the **/etc/X11** directory, a typical output from **ls -l** looks like the following:

```
total 84
lrwxrwxrwx 1 root root 20 Jul 15 20:32 X -> /usr/bin/X11/XFree86
-rw-r--r-- 1 root root 2878 Jul 16 14:50 XF86Config-4 drwxr-xr-x 2
root root 4096 Jul 15 20:32 Xresources -rwxr-xr-x 1 root root 3456
Jun 1 01:59 Xsession drwxr-xr-x 2 root root 4096 Jul 15 20:34
```

```
Xsession.d -rw-r--r-- 1 root root 217 Jun 1 01:59 Xsession.options
-rw----- 1 root root 771 Jul 15 20:32 Xwrapper.config drwxr-xr-x
2 root root 4096 Jul 15 20:35 app-defaults . . . lines deleted . .
.
```

This listing shows considerable information about every directory entry — each of which can be a file or another directory. Looking at a line from the right column to the left, you see that the rightmost column shows the name of the directory entry. The date and time before the name show when the last modifications to that file were made. To the left of the date and time is the size of the file in bytes.

The file's group and owner appear to the left of the column that shows the file size. The next number to the left indicates the number of links to the file. (A *link* is like a shortcut in Windows.) Finally, the leftmost column shows the file's permission settings, which determine who can read, write, or execute the file.



—The first letter has a special meaning, as the following list shows:

- ◆ **l:** The file is a *symbolic link* to another file. In other words, it's a shortcut to something else.
- ◆ **d:** The file is a directory. It will appear as a folder in a GUI.
- ◆ **- (hyphen):** The file is normal. By *normal*, I mean that it isn't a directory, a link, or anything else odd. Most of the items on your system are just normal files.
- ◆ **b:** The file represents a block device, such as a disk drive.
- ◆ **c:** The file represents a character device, such as a serial port or a terminal.



After that first letter, the leftmost column shows a sequence of nine characters, which appear as **rw-rw-rw-** when each letter is present. Each letter indicates a specific permission. A hyphen (-) in place of a letter indicates no permission for a specific operation on the file. Think of these nine letters as three groups of three letters (**rw-**), interpreted as follows:

- ◆ **Leftmost **rw-** group:** Controls the read, write, and execute permission of the file's owner. In other words, if you see **rw-** in this position, the file's owner can read (**r**), write (**w**), and execute (**x**) the file. A hyphen in the place of a letter indicates no permission. Thus the string **rw-** means the owner has read and write permission but not execute permission. Although executable programs (including shell programs) typically have execute permission, directories treat execute permission as equivalent to *use* permission: A user must have execute permission on a directory before he or she can open and read the contents of the directory.
- ◆ **Middle **rw-** group:** Controls the read, write, and execute permission of any user belonging to that file's group.
- ◆ **Rightmost **rw-** group:** Controls the read, write, and execute permission of all other users (collectively thought of as *the world*).

Thus, a file with the permission setting **rw-----** is accessible only to the file's owner, whereas the permission setting **rw-r--r--** makes the file readable by the world.

An interesting feature of the **ls** command is that it doesn't list any file whose name begins with a period. To see these files, you must use the **ls** command with the **-a** option, as follows:

```
ls -a
```

Try this command in your home directory and then compare the result with what you see when you don't use the **-a** option:

1. Type **cd** to change to your home directory.
2. Type **ls -F** to see the files and directories in your home directory.
3. Type **ls -aF** to see everything, including hidden files.



Most Linux commands take single-character options, each with a hyphen as a prefix. When you want to use several options, type a hyphen and *concatenate* (string together) the option letters, one after another. Thus `ls -al` is equivalent to `ls -a -l` as well as `ls -l -a`.

## Commands for changing permissions and ownerships

You may need to change a file's permission settings to protect it from others. Use the **chmod** command to change the permission settings of a file or a directory.

To use **chmod** effectively, you have to specify the permission settings. A good way is to concatenate letters from the columns of Table 4-4 in the order shown (Who/Action/Permission).

Book II  
Chapter 4

Navigating the  
LinuxFileSystem



| Table 4-4 Letter Codes for File Permissions |                        |                    |
|---------------------------------------------|------------------------|--------------------|
| Who                                         | Action                 | Permission         |
| <b>u</b> (user)                             | <b>+</b> (add)         | <b>r</b> (read)    |
| <b>g</b> (group)                            | <b>-</b> (remove)      | <b>w</b> (write)   |
| <b>o</b> (others)                           | <b>=</b> (assign)      | <b>x</b> (execute) |
| <b>a</b> (all)                              | <b>s</b> (set user ID) |                    |



You use only the single character from each column — the text in parentheses is for explanation only.

For example, to give everyone read access to all files in a directory, pick **a** (for *all*) from the first column, **+** (for *add*) from the second column, and **r** (for *read*) from the third column to come up with the permission setting **a+r**. Then use the set of options with **chmod**, like this:

```
chmod a+r *
```

On the other hand, to permit everyone to execute one specific file, type **chmod a+x filename**

Suppose you have a file named **mystuff** that you want to protect. You can make it accessible to no one but you if you type the following commands, in this order:

```
chmod a-rwx mystuff  chmod u+rw
mystuff
```

The first command turns off all permissions for everyone, and the second command turns on the read and write permissions for the owner (you). Type `ls -l` to verify that the change took place. (You see a permission setting of **-rw-----**.)

Sometimes you have to change a file's user or group ownership for everything to work correctly. For example, suppose you're instructed (by a manual, what else?) to create a directory named **cups** and give it the ownership of user ID **lp** and group ID **sys**. How do you do it?

Well, you can log in as **root** and create the **cups** directory with the command **mkdir**:

```
mkdir cups
```



If you check the file's details with the `ls -l` command, you see that the user and group ownership is `root root`.

To change the owner, use the `chown` command. For example, to change the ownership of the `cups` directory to user ID `lp` and group ID `sys`, type `chown lp.sys cups`

## Commands for working with files

To copy files from one directory to another, use the `cp` command. If you want to copy a file to the current directory but retain the original name, use a period (.) as the second argument of the `cp` command. Thus the following command copies the `Xresources` file from the `/etc/X11` directory to the current directory (denoted by a single period):

```
cp /etc/X11/Xresources .
```

The `cp` command makes a new copy of a file and leaves the original intact.



If you want to copy the entire contents of a directory — including all subdirectories and their contents — to another directory, use the command `cp -ar sourcedir destdir`. (This command copies everything in the `sourcedir` directory to `destdir`.) For example, to copy all files from the `/etc/X11` directory to the current directory, type the following command:

```
cp -ar /etc/X11 .
```



To move a file to a new location, use the `mv` command. The original copy is gone, and a new copy appears at the destination. You can use `mv` to rename a file. If you want to change the name of `today.list` to `old.list`, use the `mv` command, as follows: `mv today.list old.list`

On the other hand, if you want to move the `today.list` file to a subdirectory named `saved`, use this command:

```
mv today.list saved
```

An interesting feature of `mv` is that you can use it to move entire directories (with all their subdirectories and files) to a new location. If you have a directory named `data` that contains many files and subdirectories, you can move that entire directory structure to `old_data` by using the following command:

```
mv data old_data
```

To delete files, use the `rm` command. For example, to delete a file named `old.list`, type the following command:

```
rm old.list
```



Be careful with the `rm` command — especially when you log in as `root`. You can inadvertently delete important files with `rm`.

## Commands for working with directories

To organize files in your home directory, you have to create new directories. Use the `mkdir` command to create a directory. For example, to create a directory named `images` in the current directory, type the following:

```
mkdir images
```

After you create the directory, you can use the `cd images` command to change to that directory.



You can create an entire directory tree by using the **-p** option with the **mkdir** command. For example, suppose your system has a **/usr/src** directory and you want to create the directory tree **/usr/src/book/java/examples/ applets**. To create this directory hierarchy, type the following command: **mkdir -p /usr/src/book/java/examples/applets**



When you no longer need a directory, use the **rmdir** command to delete it.

You can delete a directory only when the directory is empty.

To remove an empty directory tree, you can use the **-p** option, like this: **rmdir -p**

**/usr/src/book/java/examples/applets**

This command removes the empty parent directories of **applets**. The command stops when it encounters a directory that's not empty.

Book II  
Chapter 4

Navigating the  
Linux File System

## Commands for finding files

The **find** command is useful for locating files (and directories) that meet your search criteria.

When I began using Unix many years ago (Berkeley Unix in the early 1980s), I was confounded by the **find** command. I stayed with one basic syntax of **find** for a long time before graduating to more complex forms. The basic syntax that I discovered first was for finding a file anywhere in the file system. Here's how it goes: Suppose you want to find any file or directory with a name that starts with **gnome**. Type the following **find** command to find these files:

```
find / -name "gnome*" -print
```

If you're not logged in as **root**, you may get a bunch of error messages. If these error messages annoy you, just modify the command as follows and the error messages are history. (Or, as Unix aficionados say, "Send 'em to the bit bucket.")

```
find / -name "gnome*" -print 2> /dev/null
```

This command tells **find** to start looking at the **root** directory (**/**) to look for filenames that match **gnome\*** and to display the full pathname of any matching file. The last part (**2> /dev/null**) simply sends the error messages to a special file that's the equivalent of simply ignoring them.